



Industrial Machine Learning on Hadoop and Spark

Maks Nakhodnov, Bremen 2025

Distributed computing. MapReduce. YARN

Plan:

How to process big data?

- Hadoop Streaming
- Word Count Example
- MapReduce v1.0, YARN

MapReduce. Hadoop Streaming

- Hadoop MapReduce has native implementations for:
 - Java, C++, Scala
- No Python API
- Hadoop Streaming — a wrapper for the Java API for Python code
- Downside: purely textual data transfer

MapReduce. Hadoop Streaming

- Mapper, Reducer, ... receive the input from *stdin* and write to *stdout*
- Mapper processes input line-by-line and writes the output KV pair, separating using <TAB>
- Reducer receives sorted by key KV pairs (instead of the values list)
- Roughly equal to:
`$cat input | python map.py | sort | python reduce.py`
- All of the 9 parts of the MapReduce workflow can be adjusted using command-line keys
- <https://hadoop.apache.org/docs/r1.2.1/streaming.html>

MapReduce. Word Count I

Count the number of occurrences of each word in the text

corpus: $D = \{d_1, \dots, d_n\} \implies \{(w_1, c_1), \dots, (w_p, c_p)\}$

The simplest implementation:

The problem is that too much data is being transferred between the map and reduce phases.

```
def map(doc_id, doc):  
    for word in doc:  
        Emit(word, 1)  
  
def reduce(word, [c1, c1, ...]):  
    count = 0  
    for c in [c1, c2, ...]:  
        count += c  
    Emit(word, count)
```

MapReduce. Word Count II. Combiner

Count the number of occurrences of each word in the text

corpus: $D = \{d_1, \dots, d_n\} \implies \{(w_1, c_1), \dots, (w_p, c_p)\}$

The Combiner function is optional and can be used to optimize the process by reducing the amount of data transferred between the map and reduce phases:

```
def combiner(word, [c1, c2, ...]):  
    count = 0  
    for c in [c1, c2, ...]:  
        count += c  
    Emit(word, count)
```

MapReduce. Word Count III. In-mapper combiner

Count the number of occurrences of each word in the text

corpus: $D = \{d_1, \dots, d_n\} \implies \{(w_1, c_1), \dots, (w_p, c_p)\}$

The next problem is that the map function is emitting too many intermediate key-value pairs.

We can optimize the map function by using a local aggregation technique:

```
def map(doc_id, doc):  
    H = defaultdict(int)  
    for word in doc:  
        H[word] += 1  
    for word, count in H.items():  
        Emit(word, count)
```

MapReduce. Word Count III. In-mapper combiner

Count the number of occurrences of each word in the text corpus: $D = \{d_1, \dots, d_n\} \implies \{(w_1, c_1), \dots, (w_p, c_p)\}$

- The number of intermediate key-value pairs emitted by the map function (+ the amount of data transferred between the map and reduce phases) is reduced
- The local aggregation should not use too much memory
- If a document is too large, we may need to flush the local aggregation to the output periodically

MapReduce. Word Count IV. In-mapper combiner V2

Count the number of occurrences of each word in the text

corpus: $D = \{d_1, \dots, d_n\} \implies \{(w_1, c_1), \dots, (w_p, c_p)\}$

If the number of distinct words is small, we can reuse the same dictionary for different documents:

We have to flush the dictionary periodically to avoid OOM

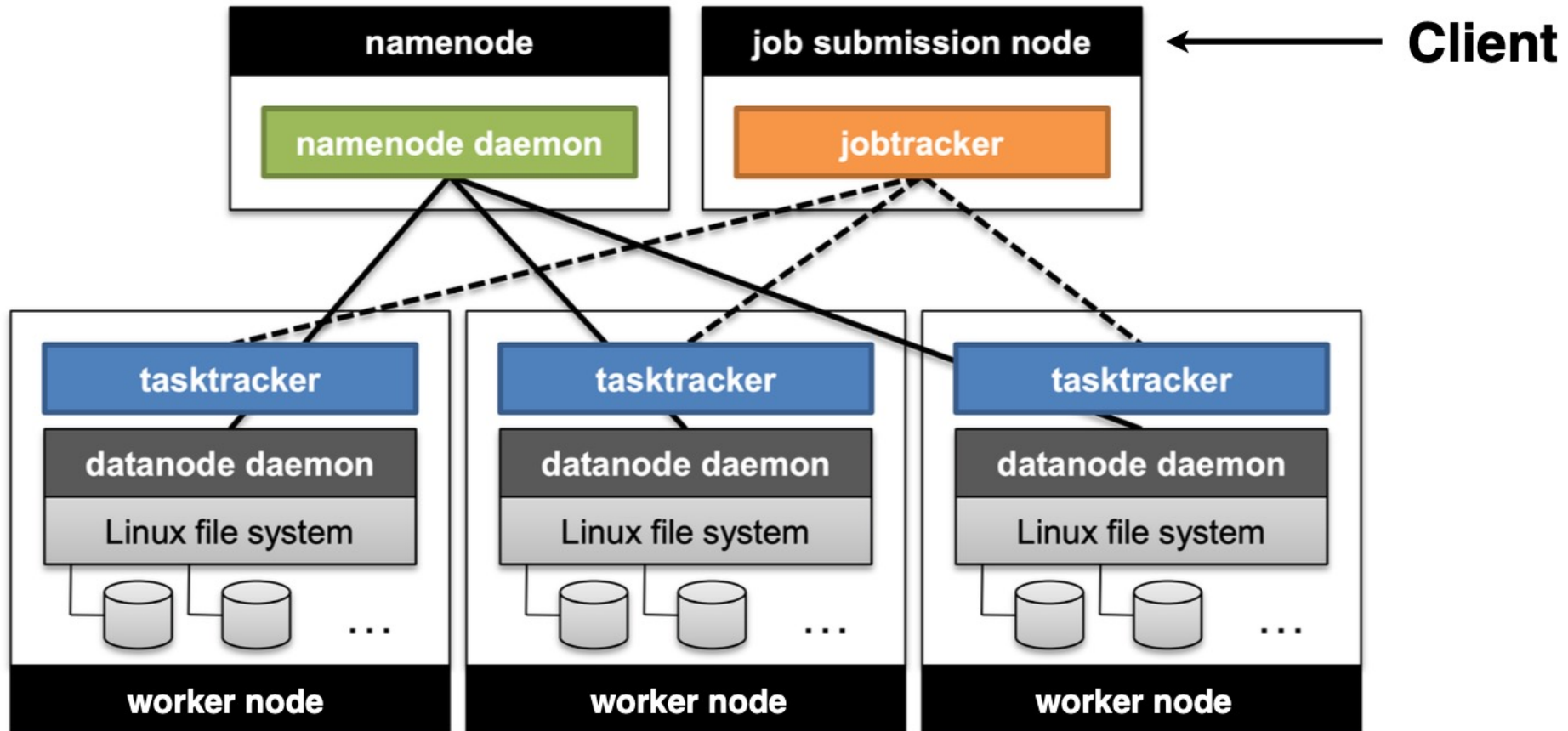
The dictionary can be initialized from local FC to implement tricky behavior (i.e., Map-side join), or be broadcast using --files CLI flag

```
# Initialize the dictionary outside the map function.
H = defaultdict(int)
# The map function now reuses the same dictionary for different documents.
def map(doc_id, doc):
    for word in doc:
        H[word] += 1
# Close the map function by emitting the contents of the dictionary.
for word, count in H.items():
    Emit(word, count)
```

Hadoop. Resource Management

1. How many mappers and reducers should be launched, when, and where?
2. What resources should be allocated for task execution?
3. How can the execution of the task be monitored?
4. How should the executable code be delivered to the workers?
5. How should the task execution process be logged?
6. What actions should be taken in case of failures?

Hadoop. MapReduce V1



Hadoop. MapReduce V1. JobTracker

- Manages task execution and determines on which TaskTracker a worker will run
- Controls the MapReduce job execution process
- Monitors task progress
- Restarts failed or slow tasks

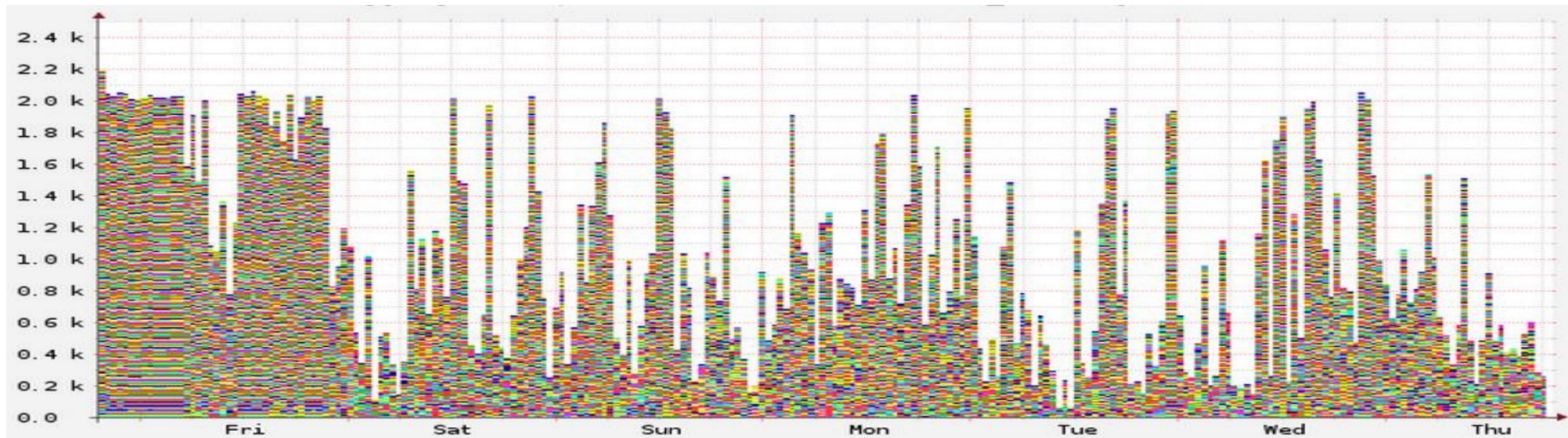
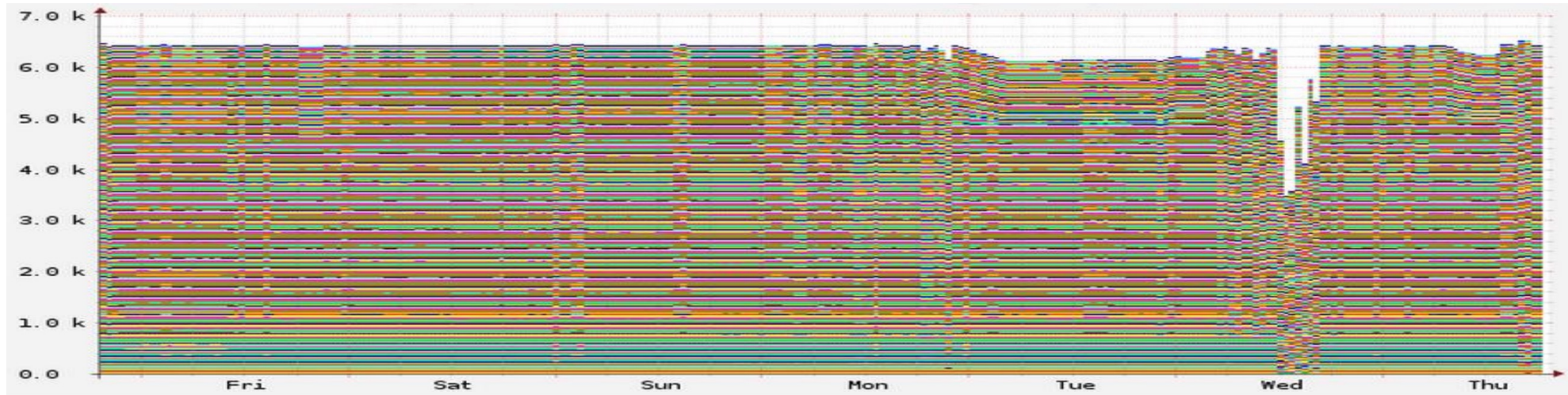
Hadoop. MapReduce V1. TaskTracker

- Manages all workers on a single server
- Receives information from the JobTracker about which worker to run on which data
- Sends progress statistics (counters) to the JobTracker
- Reports to the JobTracker on successful completion or failure of workers

Hadoop. MapReduce V1. Resource distribution

- The main drawback is the rigid resource distribution between the mapper and the reducer nodes
- The slots system:
 - A task runs in one slot
 - Each TaskTracker has a defined number of slots
 - Each slot has a defined amount of RAM allocated
 - Mappers + Reducers = N slots
- The distribution can be changed only on the cluster restart
- Limited horizontal scaling:
 - 4000 nodes
 - 40000 parallel tasks

Hadoop. MapReduce V1. Resource distribution



Hadoop. MapReduce V1. Rigid distribution

- Example:
 - 10 nodes, 16 cores each
 - TT requires 1 core
 - 300 mappers, each mapper works for 10 minutes
 - The single reducer finishes in 100 minutes
- How to choose the number of mappers and reducers?

$$100 \text{ Mapper} + 50 \text{ Reducer} \Rightarrow T^* = \frac{300}{100} * 10 + \frac{100}{50} \approx 32 \text{ minutes}$$

Hadoop. MapReduce V1. Per task distribution

- Example:
 - 10 nodes, 16 cores each
 - TT requires 10 cores
 - 300 mappers, each mapper works for 10 minutes
 - The single reducer finishes in 100 minutes
- How to choose the number of mappers and reducers?

$$\begin{cases} x \text{ Mapper} + y \text{ Reducer} = 150 \\ \frac{300}{x} * 10 + \frac{100}{y} \rightarrow \min_{x,y} \end{cases} \Rightarrow T^* \approx 28 \text{ minutes}$$

Hadoop. MapReduce V1. No distribution

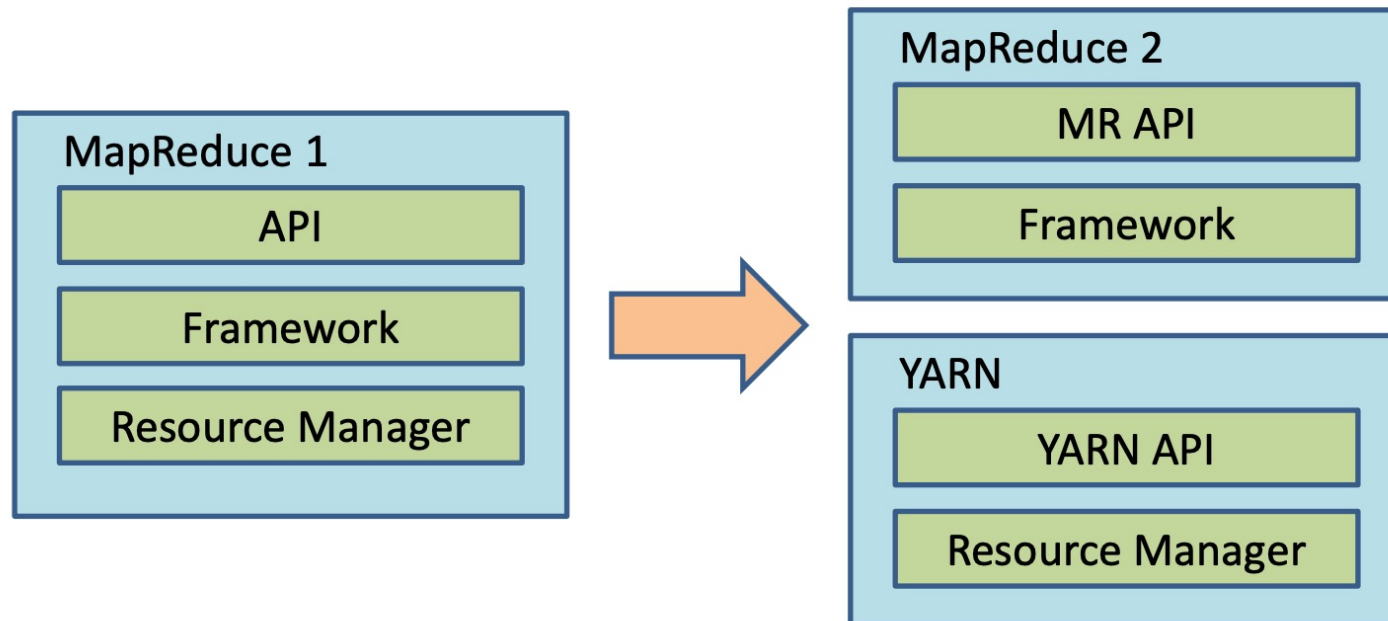
- Example:
 - 10 nodes, 16 cores each
 - TT requires 10 cores
 - 300 mappers, each mapper works for 10 minutes
 - The single reducer finishes in 100 minutes
 - How to choose the number of mappers and reducers?

$$T^* = \frac{300}{150} * 10 + \frac{100}{150} \approx 20 \text{ minutes}$$

The solution — YARN

Hadoop. YARN

- Takes over resource management functions
- Can run different types of tasks (*MapReduce is one of the options*)



Hadoop. YARN. Resource Manager

- Runs on a dedicated server
- Manages global resource allocation
 - Manages nodes
 - Manages containers
- Resolves conflicts between competing applications
 - Interacts with the ApplicationMaster
 - Doesn't know the application type
- Responsible for data protection



Hadoop. YARN. Node Manager

- Runs on all nodes in the cluster
- Interacts with the ResourceManager
- Manages processes inside containers
- Logs application data
- Maintains ACL-based security at the node level

Node
Manager



Node
Manager

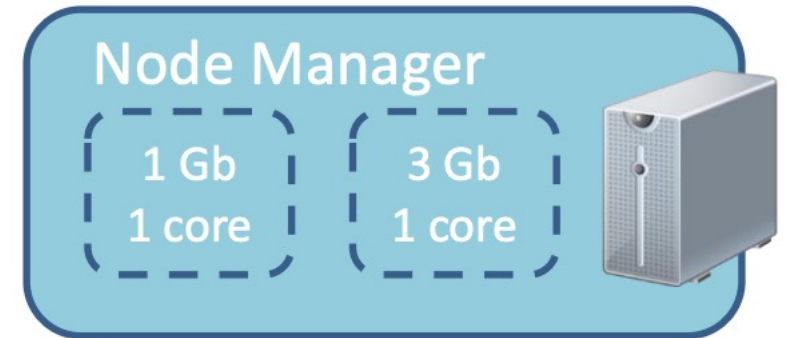


Node
Manager



Hadoop. YARN. Containers

- Created by RM on demand
- Reserves a specific amount of resources on the node (memory, CPU)
- Application runs in one or multiple containers

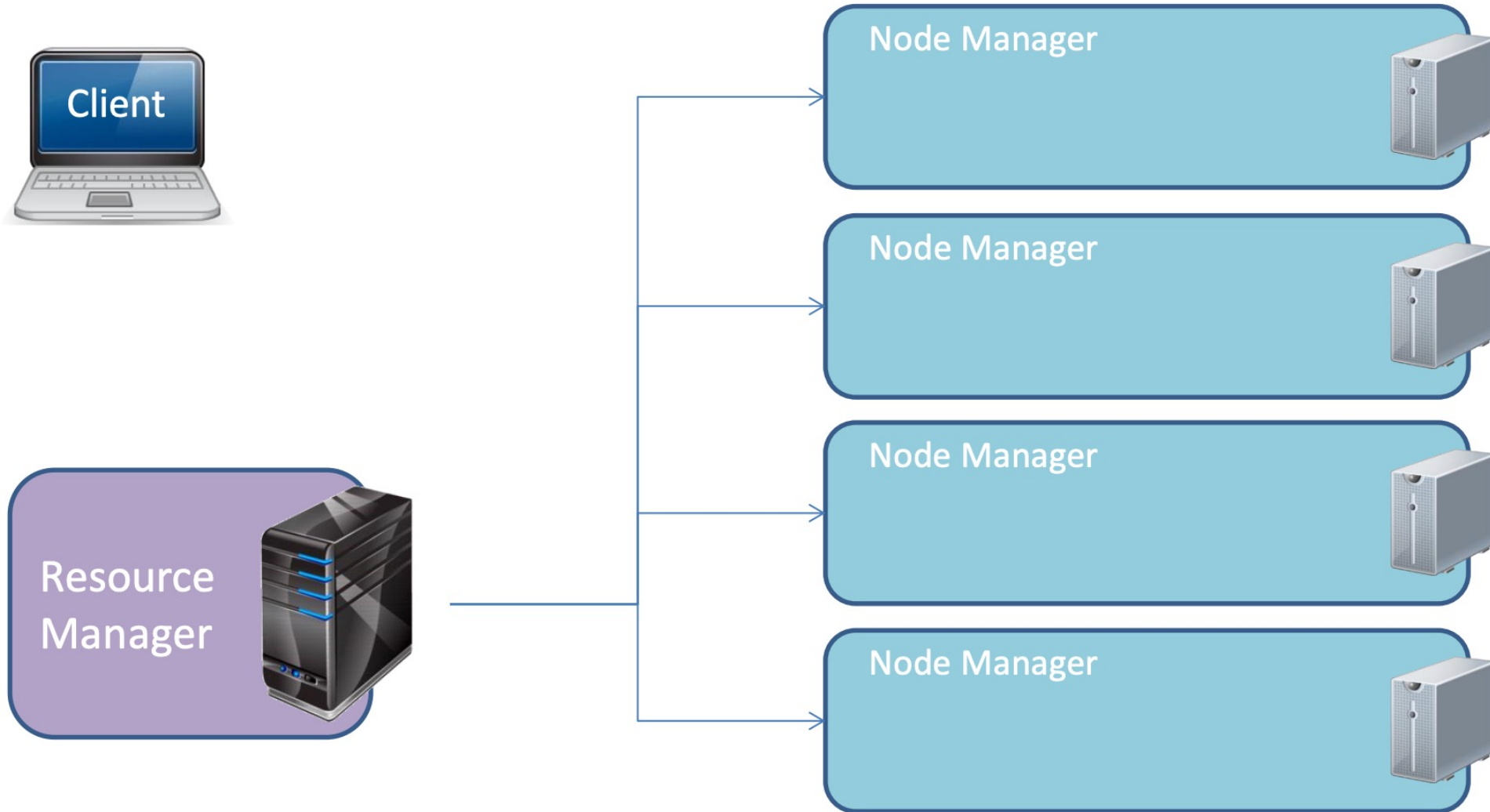


Hadoop. YARN. Application Master

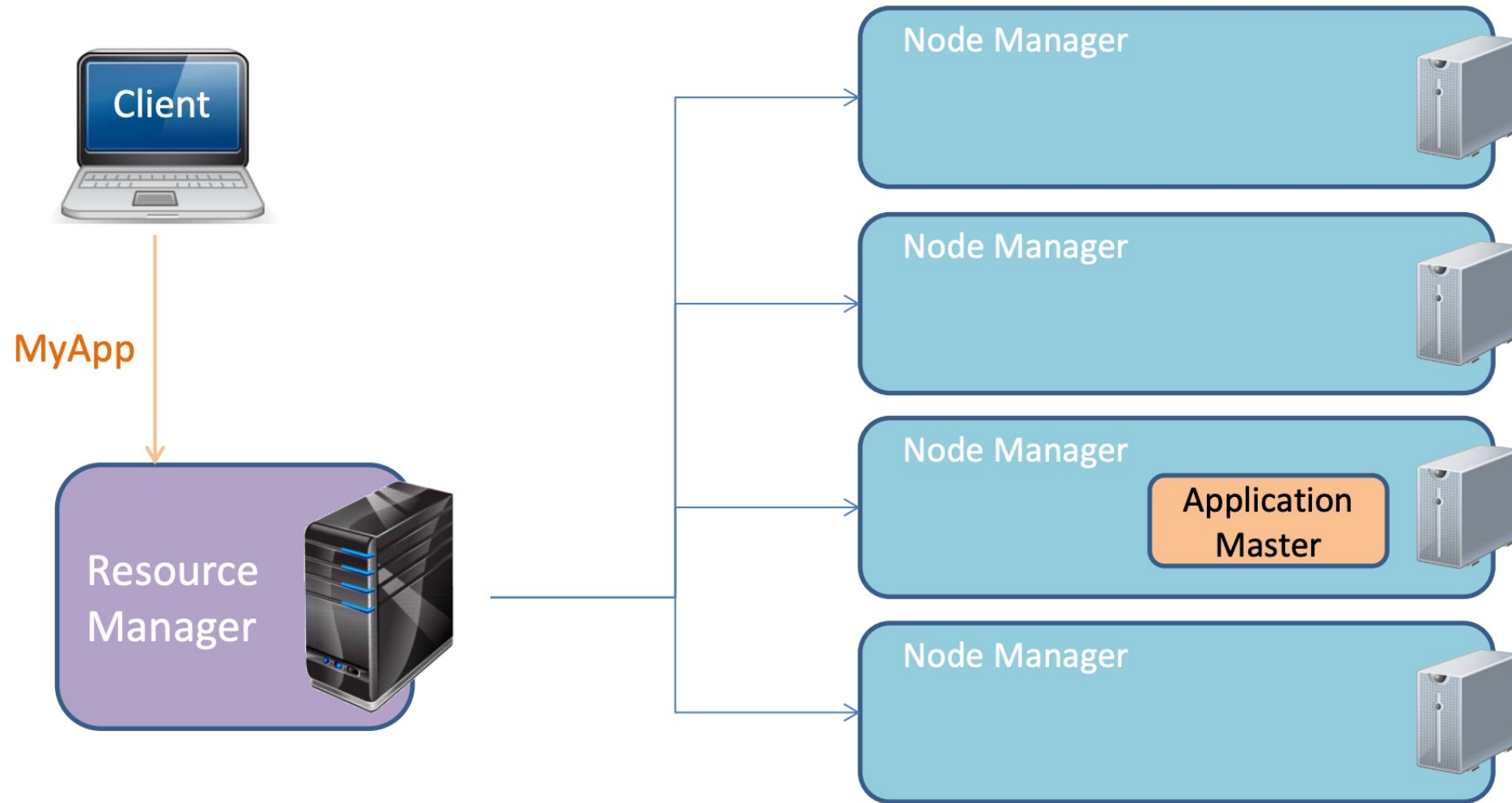
- One per application
- Depends on the type of task (has a dedicated one for MapReduce jobs)
- Runs inside a container
- Requests additional containers to execute the application



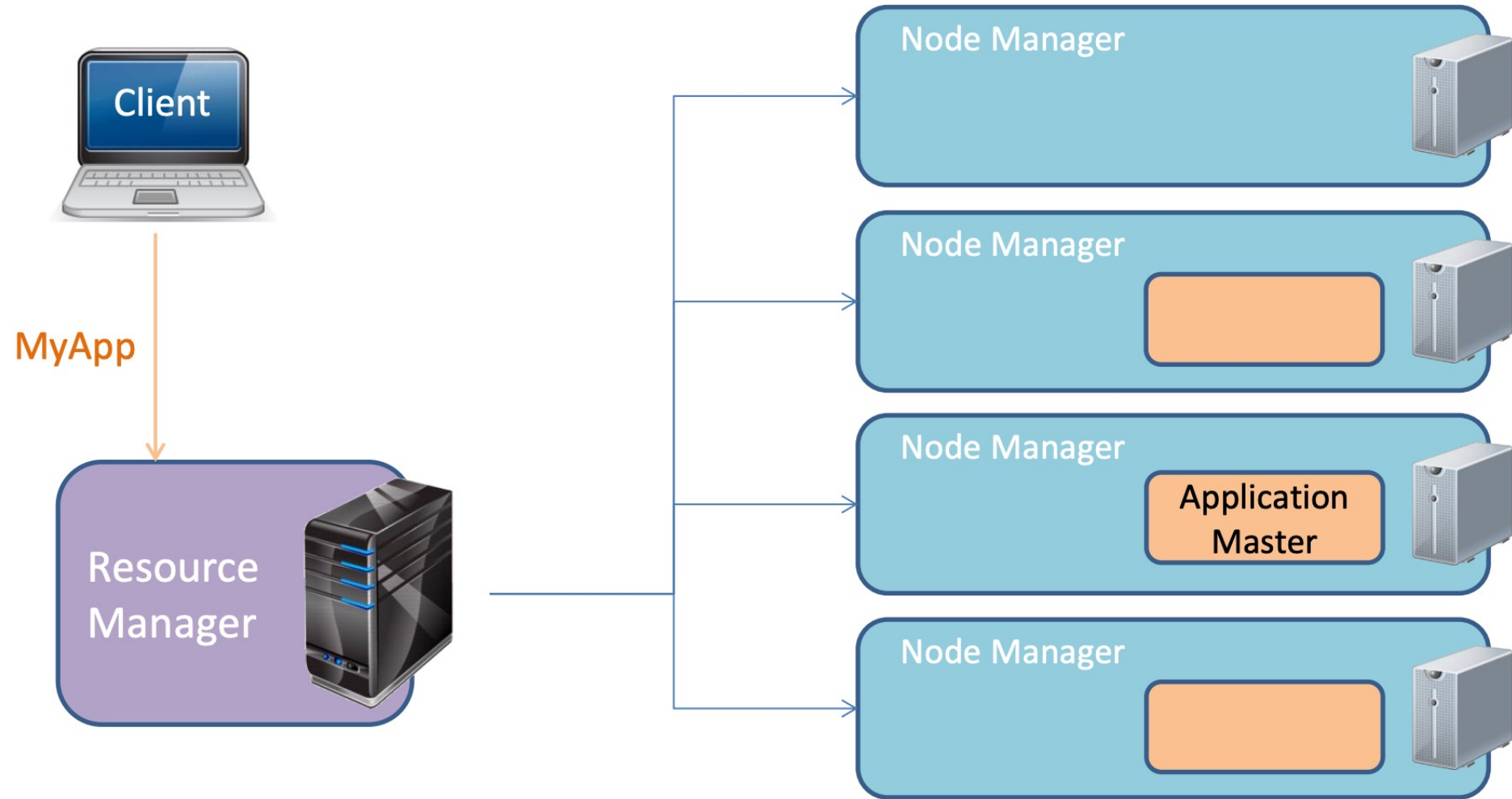
Hadoop. YARN. Workflow



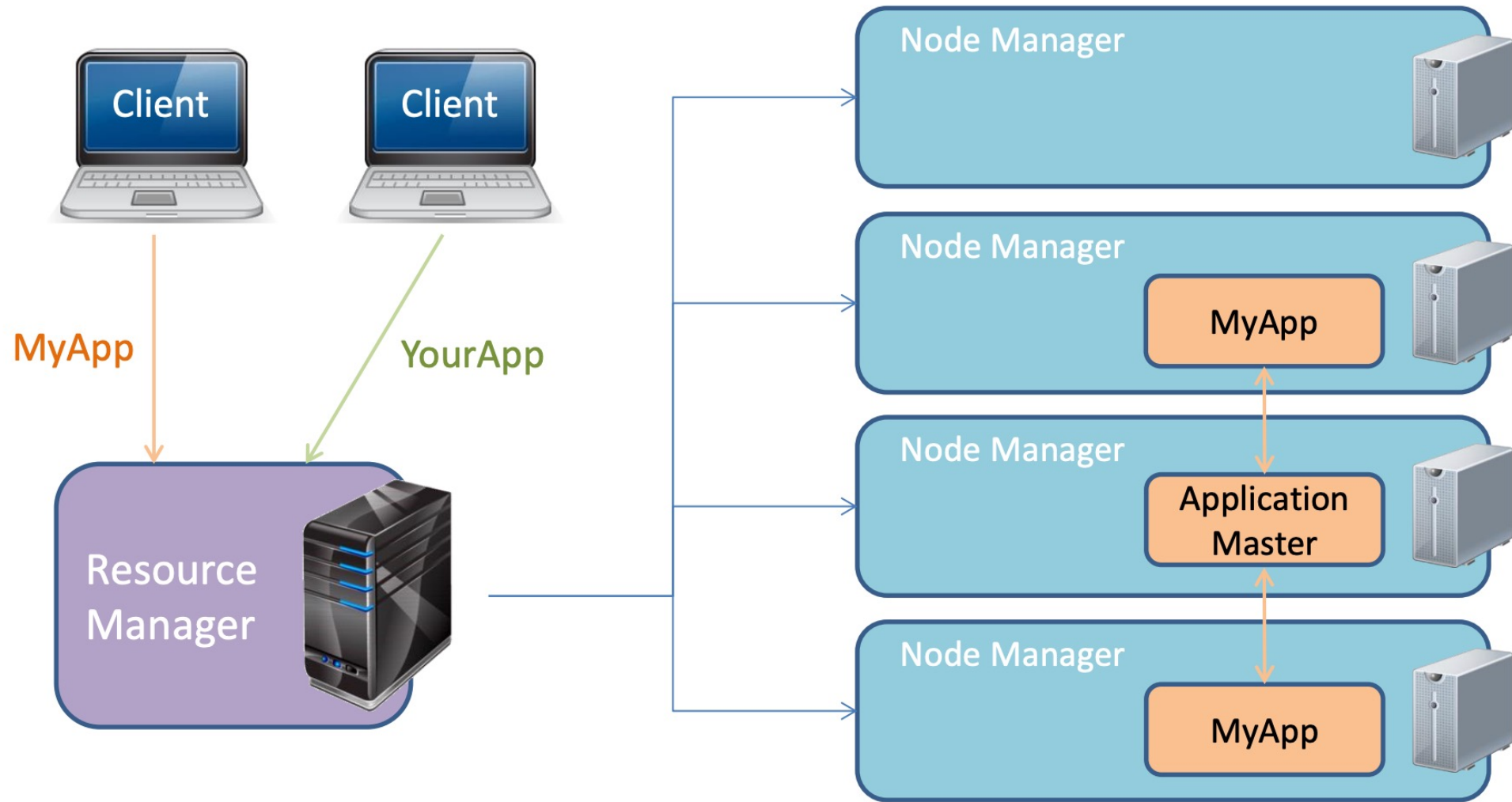
Hadoop. YARN. Workflow



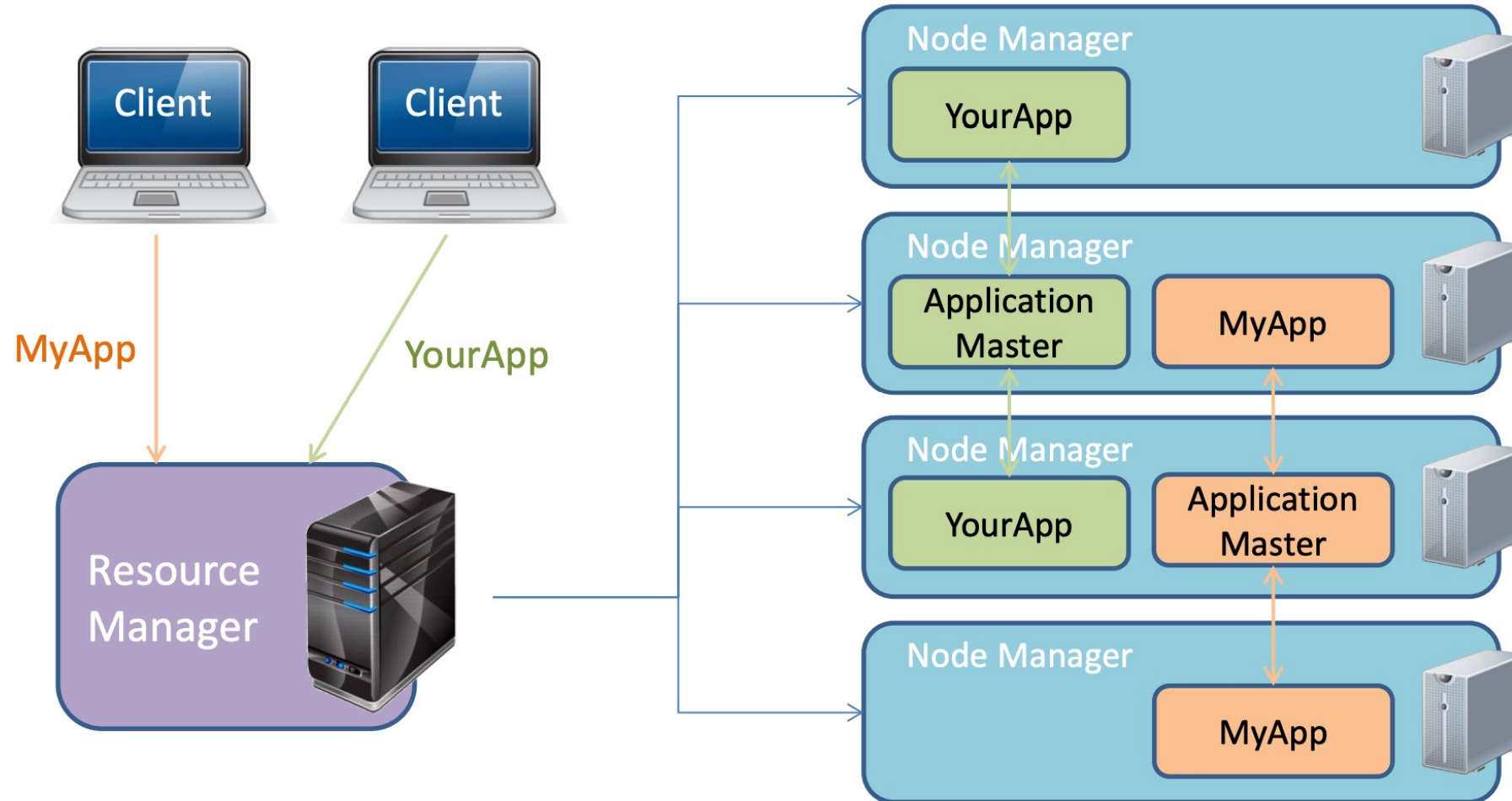
Hadoop. YARN. Workflow



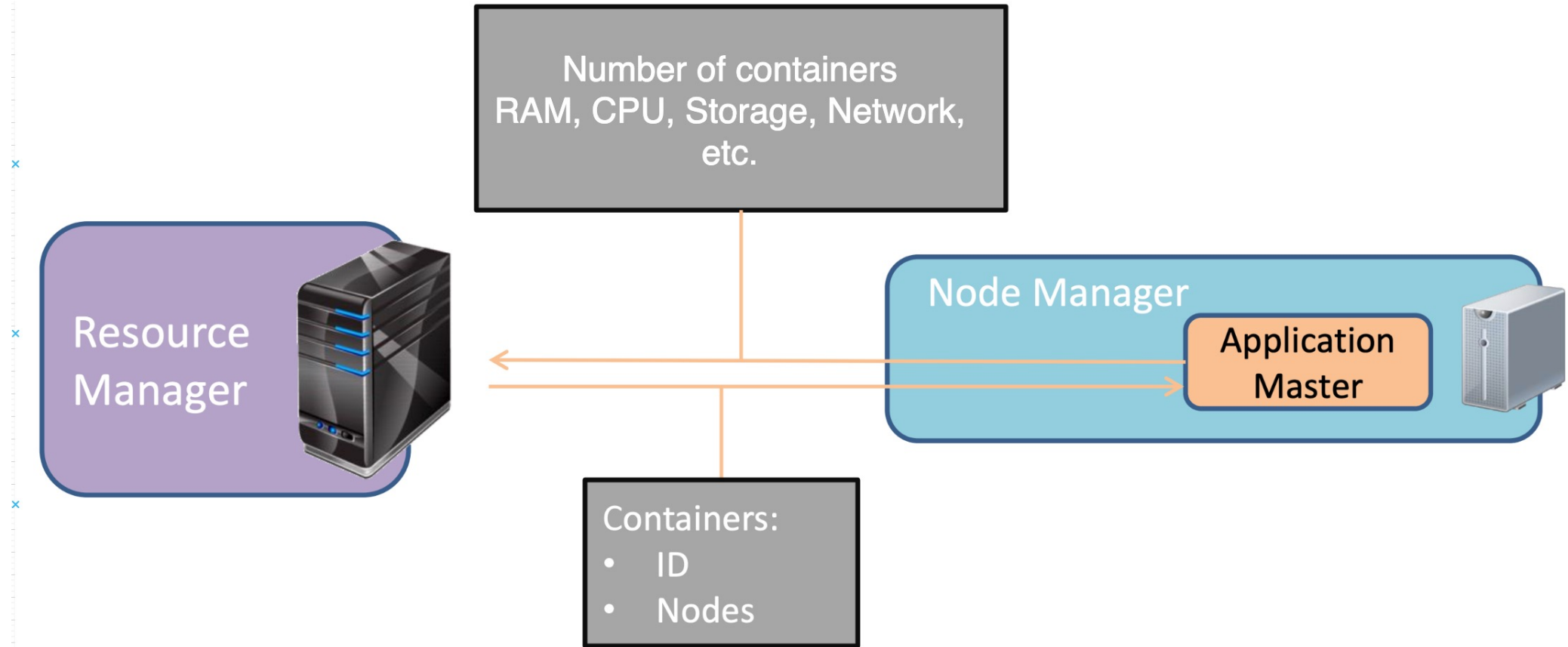
Hadoop. YARN. Workflow



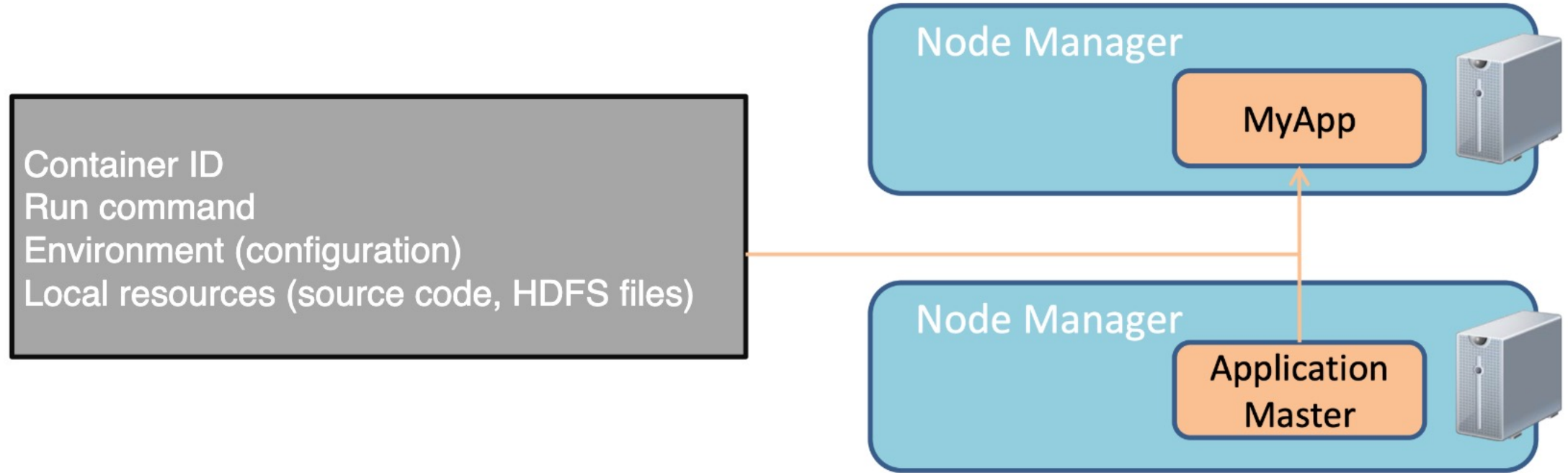
Hadoop. YARN. Workflow



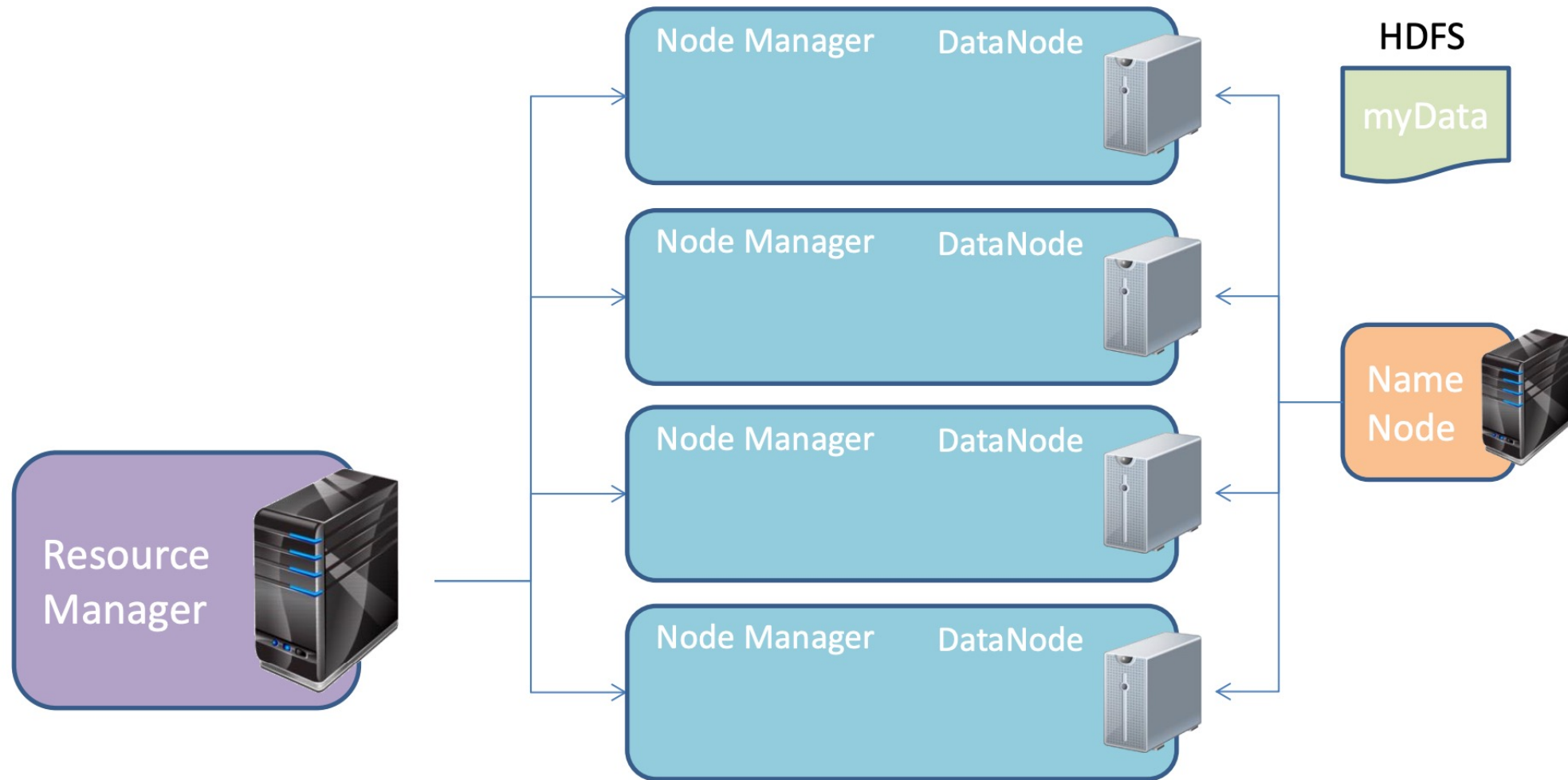
Hadoop. YARN. Resource allocation



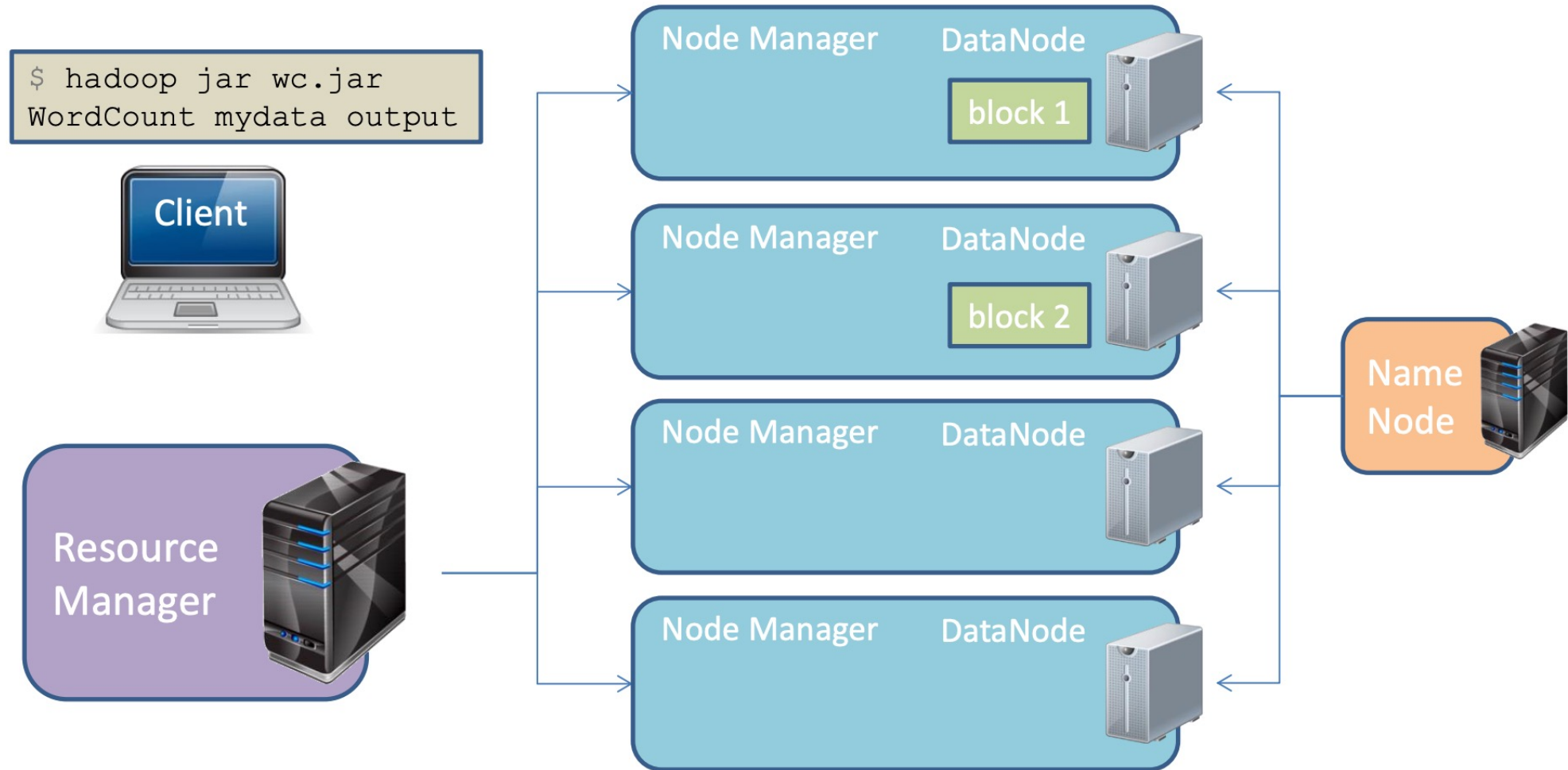
Hadoop. YARN. Container allocation



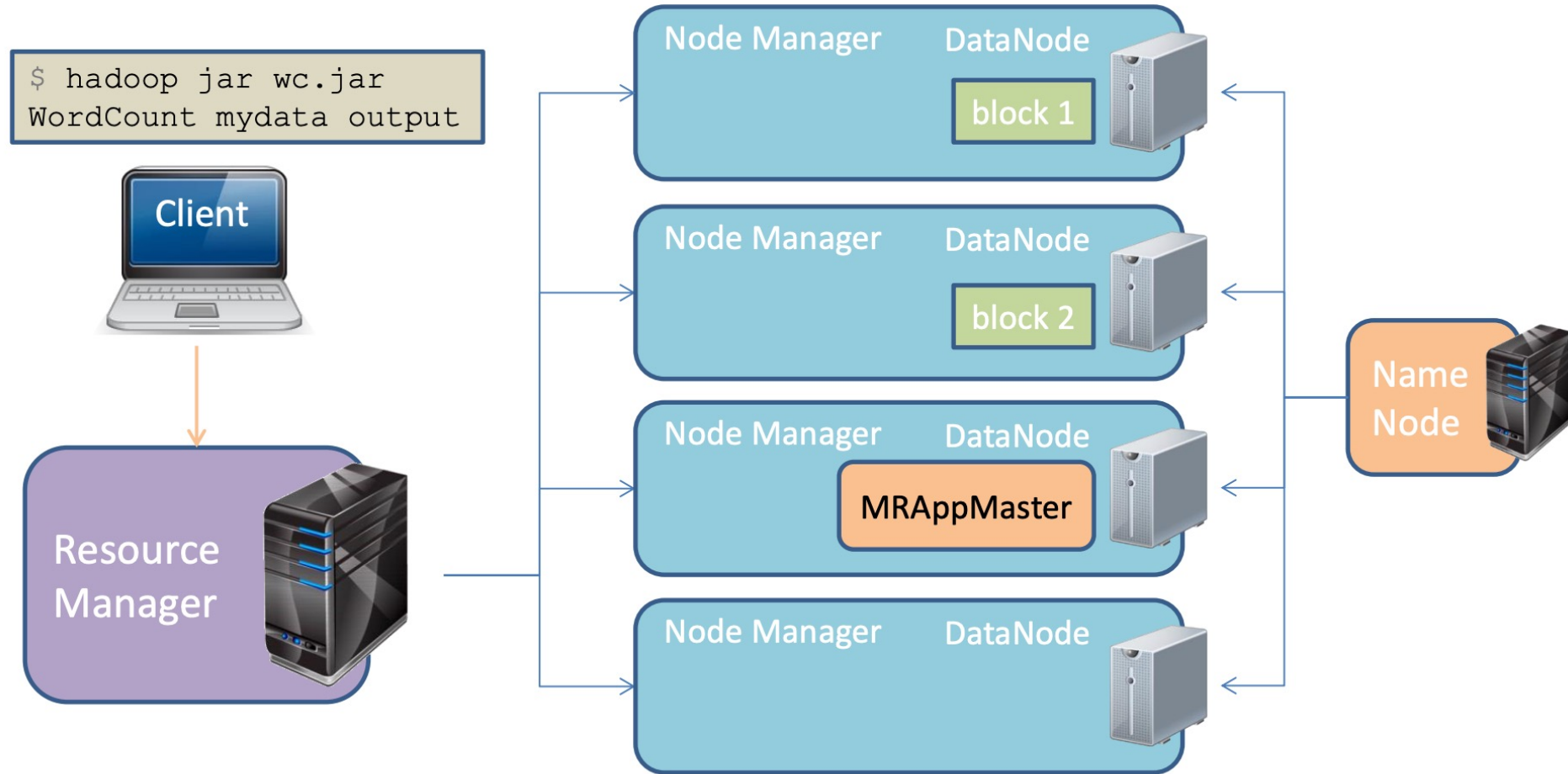
Hadoop. YARN. MapReduce



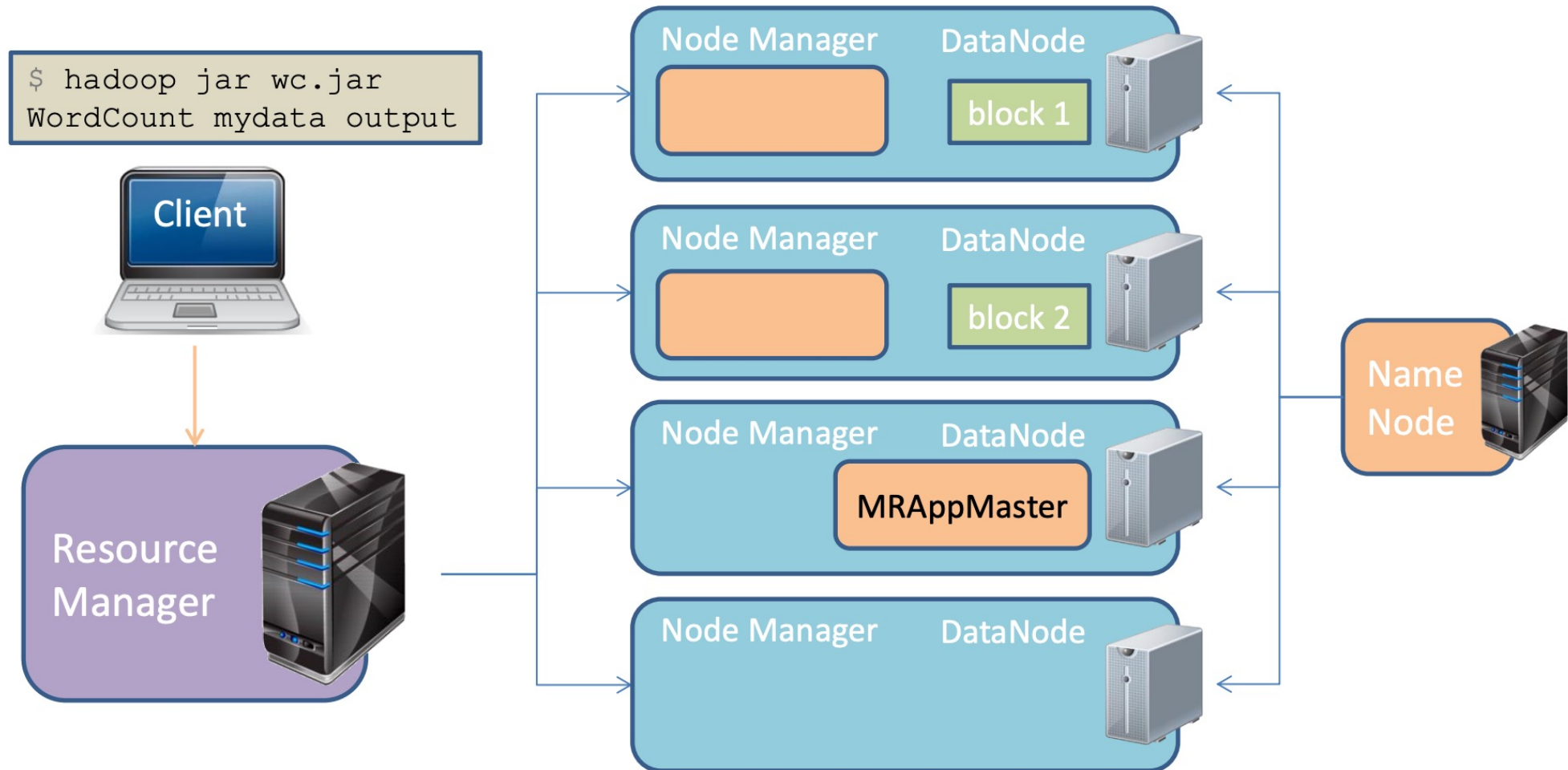
Hadoop. YARN. MapReduce



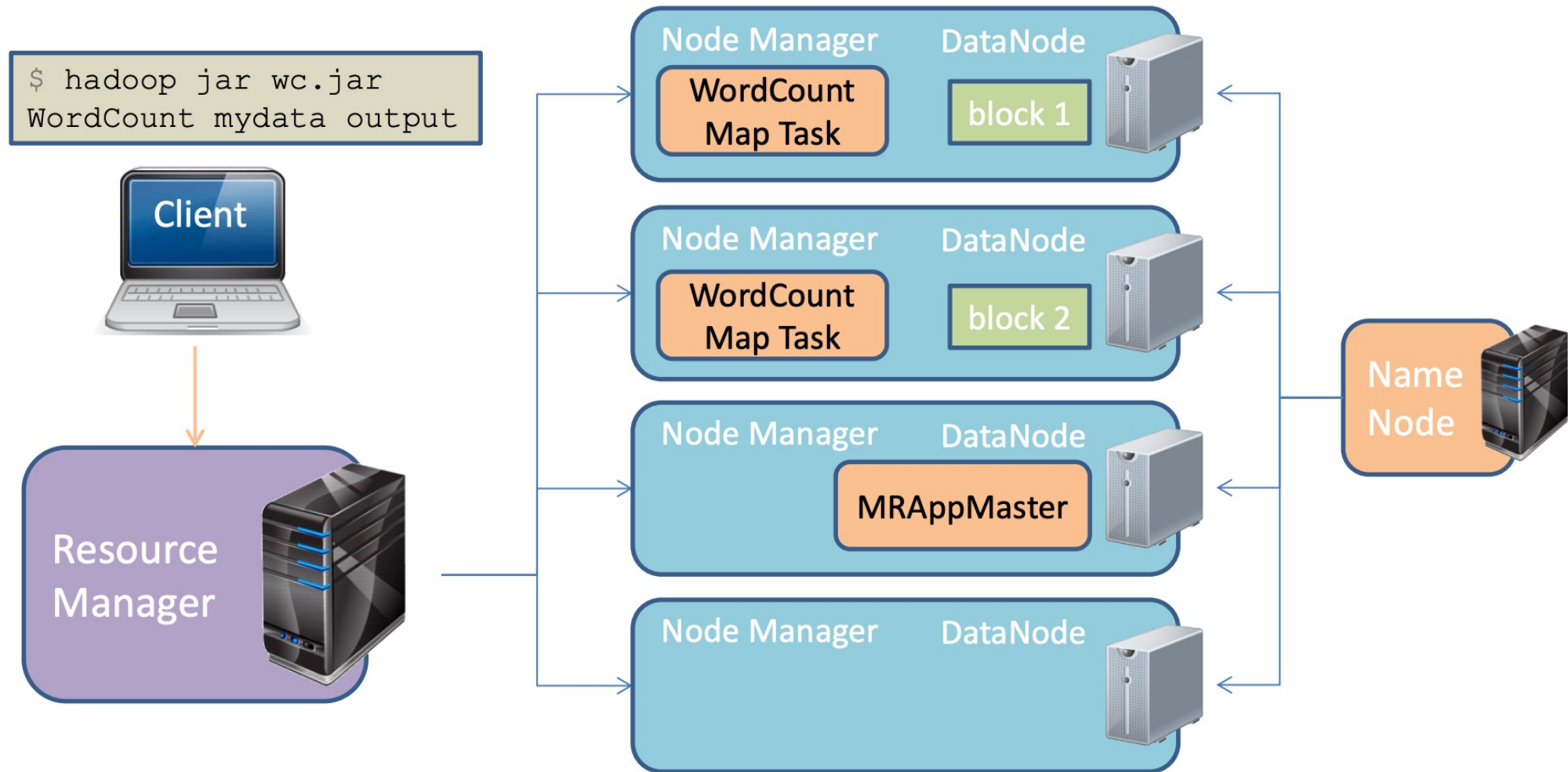
Hadoop. YARN. MapReduce



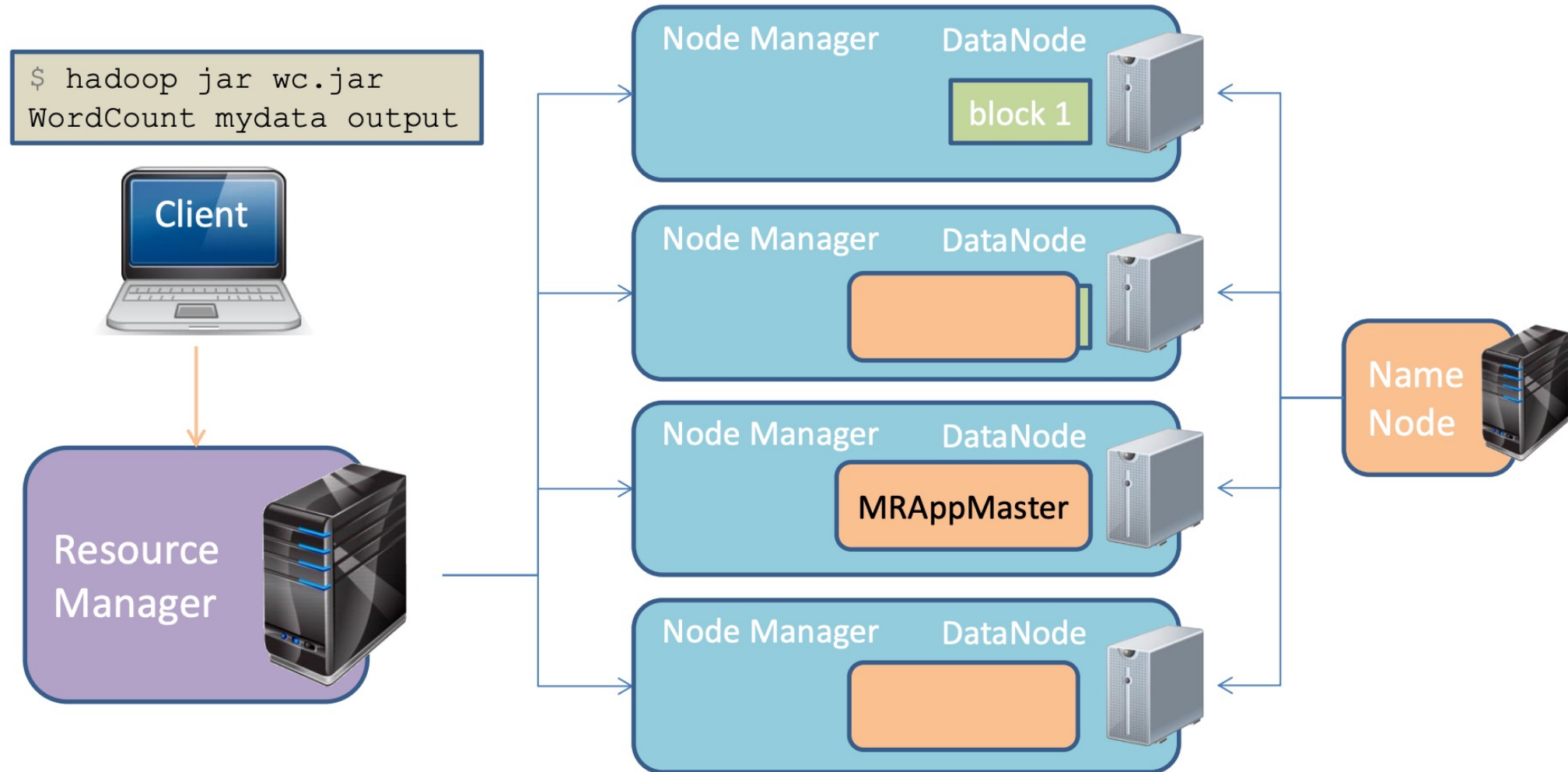
Hadoop. YARN. MapReduce



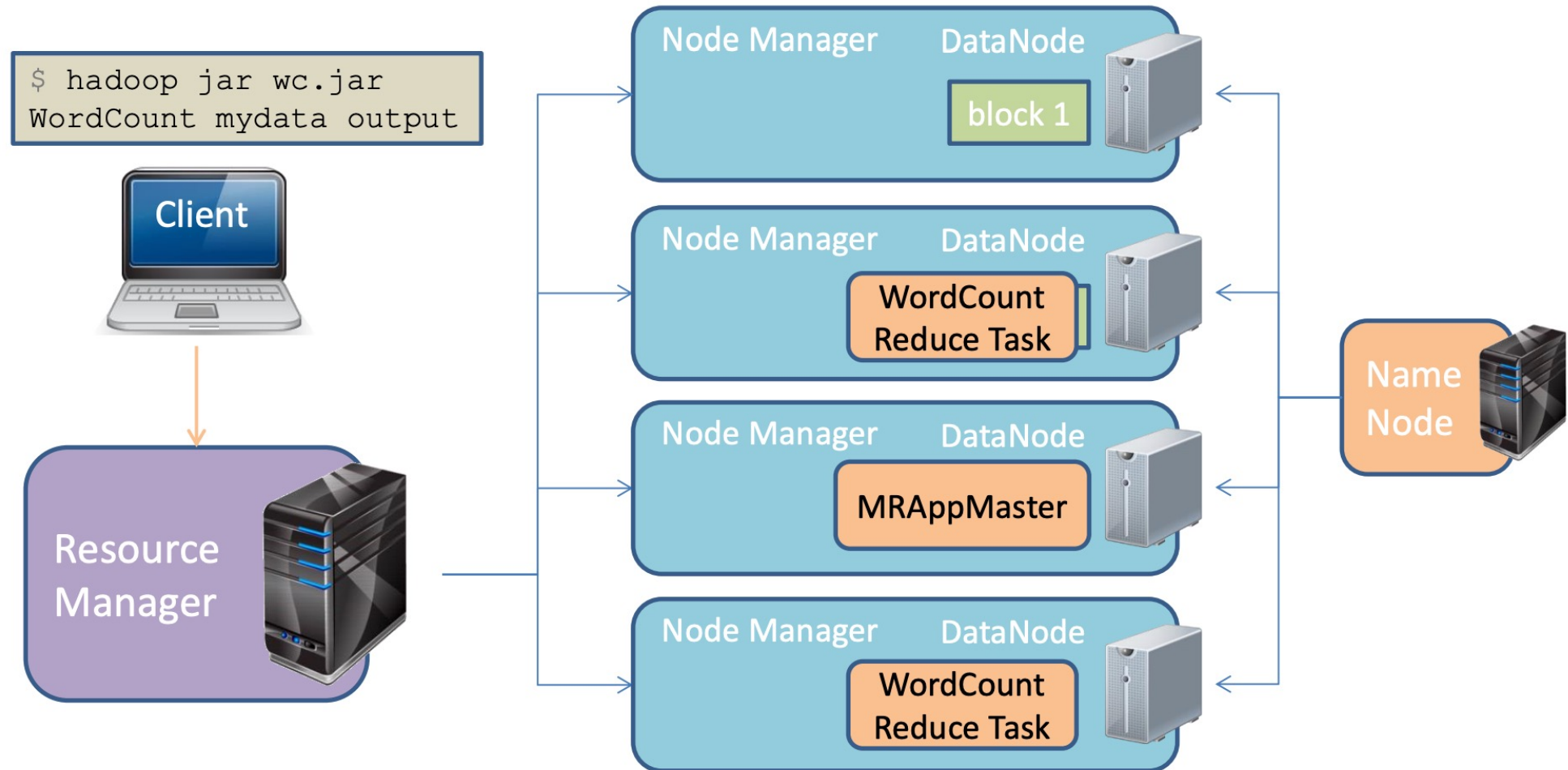
Hadoop. YARN. MapReduce



Hadoop. YARN. MapReduce

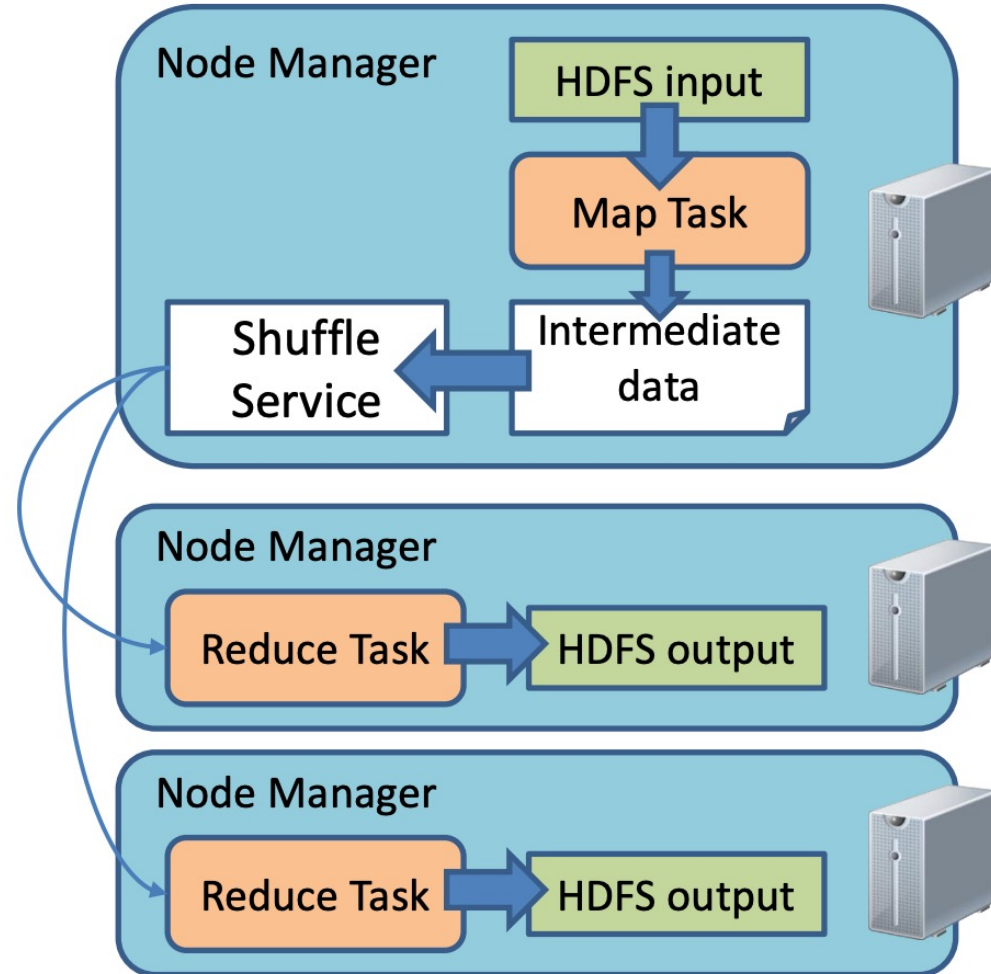


Hadoop. YARN. MapReduce



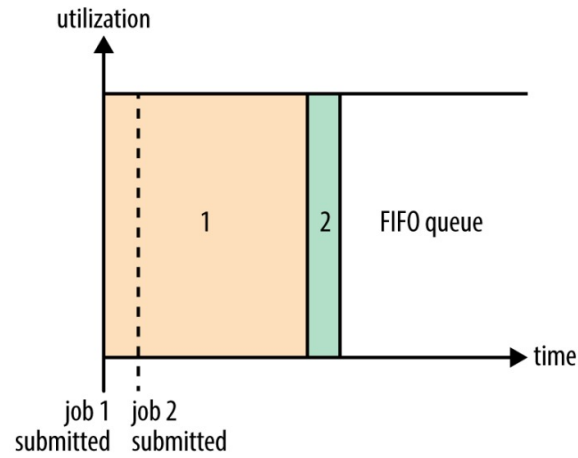
Hadoop. YARN. MapReduce. Shuffle Service

- A separate **NodeManager** process on the node that remains available even if the Mapper has finished execution
- Without the shuffle service, local Map output files could be unavailable —> the Reduce task will fail.
- Responsibilities of the shuffle service:
 - Access authorization
 - Maintaining a cache on the local disk
 - Streaming large files

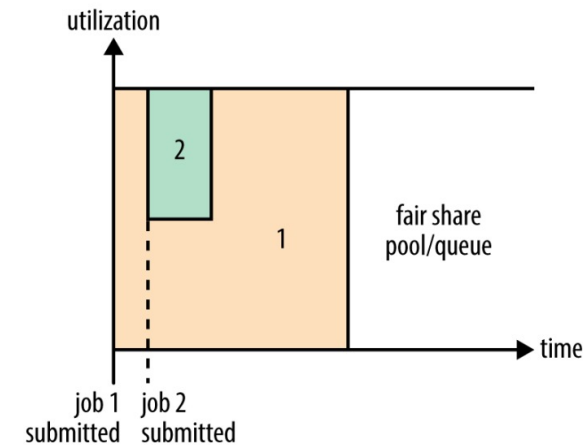


Hadoop. YARN. Task Scheduler

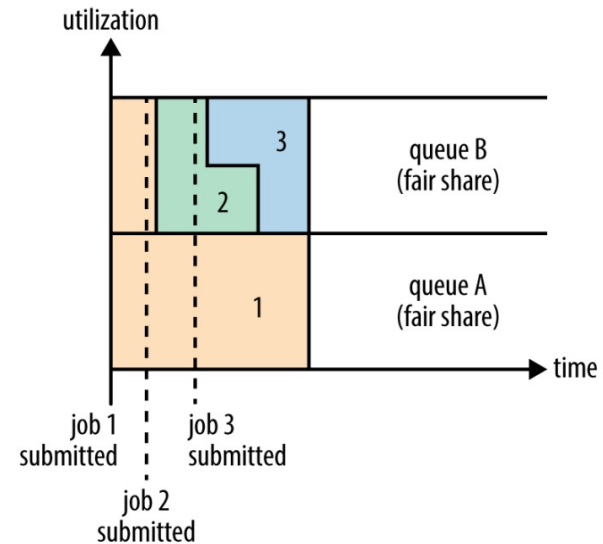
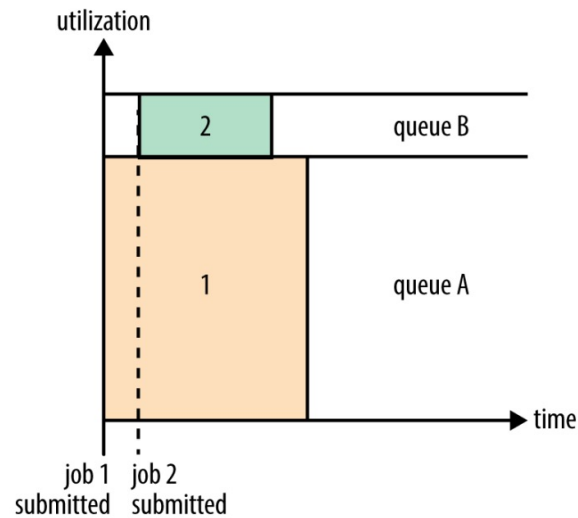
i. FIFO Scheduler



iii. Fair Scheduler



ii. Capacity Scheduler



Hadoop. YARN. Memory management

Resource Manager allocates memory:

- Minimum: `yarn.scheduler.minimum-allocation-mb` (1024 MB)
- Maximum: `yarn.scheduler.maximum-allocation-mb` (8192 MB)
- **The allocated amount has to be a multiple of the minimum amount**

Each Node Manager has a memory limit:

- `yarn.nodemanager.resource.memory-mb` (8192 MB)
- The sum of all containers cannot exceed this limit
- NodeManager does not create a container if there is insufficient memory

Hadoop. YARN. Memory management

Container sizes for Map and Reduce tasks:

- `mapreduce.map.memory.mb` = 1536
- `mapreduce.reduce.memory.mb` = 3072

JVM Heap sizes:

- `mapreduce.map.java.opts` = `-Xmx1024m`
- `mapreduce.reduce.java.opts` = `-Xmx2560m`

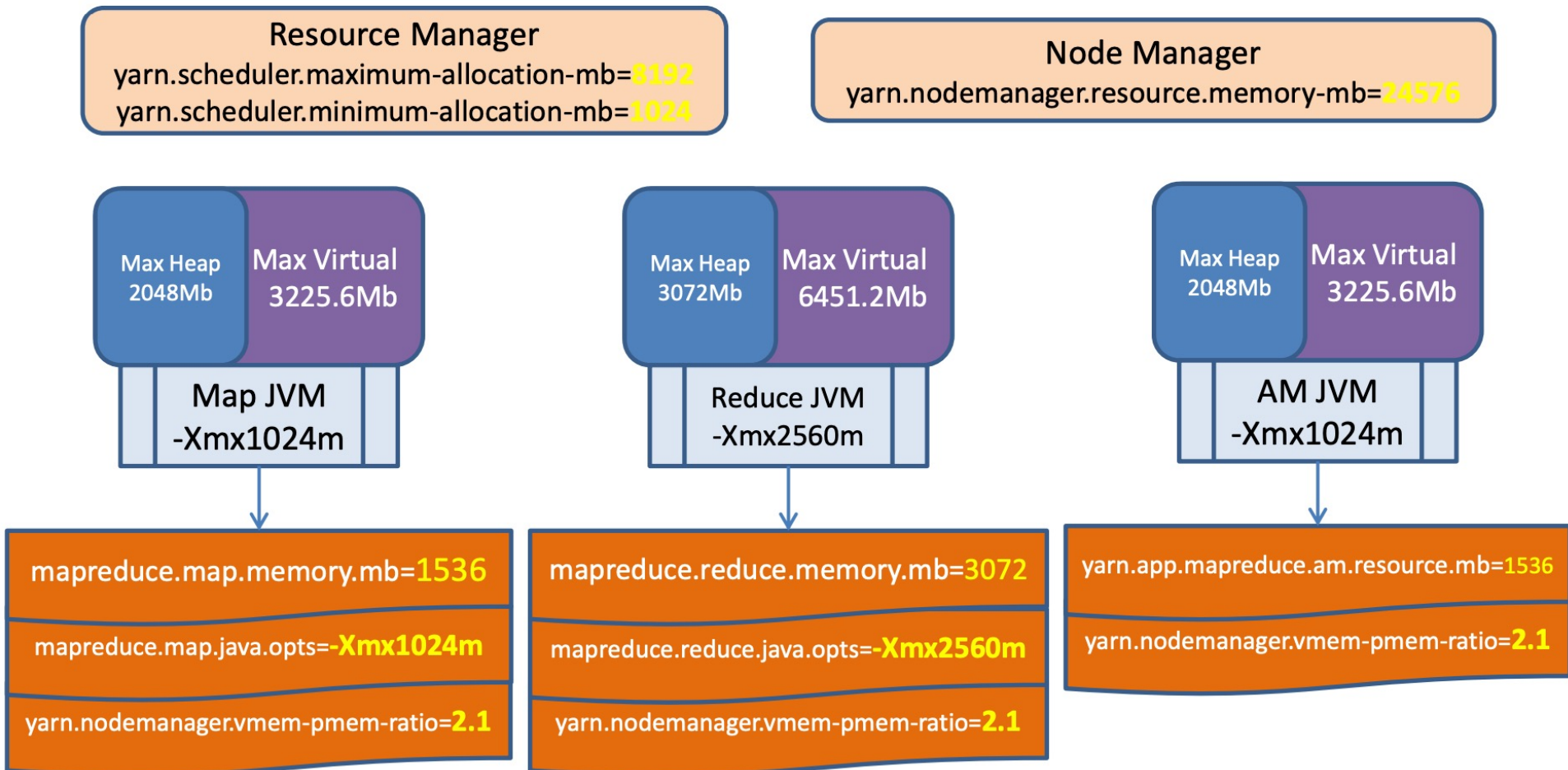
Virtual memory size is set as a ratio of physical memory:

- `yarn.nodemanager.vmem-pmem-ratio` = 2.1

If a Map or Reduce task exceeds the virtual or physical memory limit, the Node Manager kills the corresponding container:

```
Current usage: 2.1gb of 2.0gb physical memory used; 1.6gb  
of 3.15gb virtual memory used. Killing container.
```

Hadoop. YARN. Memory management



Hadoop. YARN. Fault tolerance

Failures can occur at:

- Container (Task)
- Application Manager (MRAppMaster)
- Node Manager
- Resource Manager

Hadoop. YARN. Container failure

Possible events:

- Exception
- JVM crash / server shutdown
- Process hang

Handling:

- MRAppMaster restarts the task
- Number of retries is controlled by:
 - `mapreduce.map.maxattempts` = 4
 - `mapreduce.reduce.maxattempts` = 4

Hadoop. YARN. Application Manager failure

- AM finishes with an error
- Signals from AM may not reach RM
- Resource Manager restarts the AM
 - `yarn.resourcemanager.am.max-retries = 2`
- MRAppMaster can recover task state before restart:
 - `yarn.app.mapreduce.am.job.recovery.enable = true`

Hadoop. YARN. Node Manager failure

- If Node Manager (NM) fails:
 - NM stops sending signals to RM
 - Containers on NM stop sending signals to AM
- RM removes NM from active nodes if no heartbeat for 10 minutes:
 - `yarn.resourcemanager.nm.liveness-monitor.expiry-interval-ms`
- MRAppMaster blacklists NM if 3 container failures occur:
 - `mapreduce.job.maxtaskfailures.per.tracker`

Hadoop. YARN. Resource Manager failure

- No tasks or applications can run
- Leads to cluster downtime
- RM High Availability through ZooKeeper (ZooKeeper Failover Controller)